

מבחן במבוא למדעי המחשב – 67101

מועד ב' 4.3.18

מורי הקורס: אביב זהר וג'ף רוזנשיין

במבחן זה שני חלקים. יש לענות על כל 10 השאלות בחלק א', ולבחור 4 מתוך 6 השאלות בחלק ב' (אין לענות על יותר מ-4 שאלות בחלק ב').

יש לענות על כל השאלות בגוף השאלון – בתוך "קופסאות התשובה" בלבד. דפי טיוטא לא יבדקו. אין להגישם. חומר מותר לשימוש בבחינה: אין. משך הבחינה: 3 שעות.

חלק א' – שאלות סגורות (40% – יש לענות על כל 10 הסעיפים, כל סעיף מזכה ב-4 נק')

בחלק זה של המבחן יש לכתוב את התשובה הסופית בלבד בתוך המשבצת המתאימה. אין צורך לנמק, ונימוקים גם לא יקראו.

1. מה תדפיס התוכנית הבאה?

```
def niceness(lst):  
    return [lst == lst[i:] + lst[:i] for i in range(len(lst))]  
  
print(niceness([1,2,1,2]))
```

2. מה תדפיס התוכנית הבאה?

```
y = 10  
def foo():  
    y = 3  
    print(y)  
    def goo(x):  
        print(y)  
    goo(2)  
  
foo()  
print(y)
```

3. מה תדפיס התוכנית הבאה?

```
amp = lambda f: lambda g: lambda x: g(f(f(x)))
my_dec = amp(lambda x: "*" + x + "*")

@my_dec
def my_print(y):
    print(y)

my_print("hello")
```

4. מה תדפיס התוכנית הבאה?

```
world1 = {1,2,3,4}
world2 = {1,2,7}
x = {item:(item in world2) for item in world1}
print(x)
```

5. מהי סיבוכיות זמן הריצה של התוכנית הבאה?

```
def f(n):
    alist = list(reversed(range(n)))
    for i in range(n):
        cv = alist[i]
        p = i
        while p > 0 and alist[p-1] > cv:
            alist[p] = alist[p-1]
            p -= 1
        alist[p] = cv
    return alist
```

6. מהי סיבוכיות זמן הריצה של התוכנית הבאה?

```
def g(n):  
    alist = [0]*(n*2)  
    sum = 0  
    for i in range(n*2):  
        alist[i] = n//2 + i  
    for i in range(n//2):  
        alist[i] = alist[n-i]  
    for i in range(len(alist)):  
        sum += alist[i]  
    return sum
```

7. מה תדפיס התוכנית הבאה?

```
def fad(ls):  
    if len(ls) == 1:  
        return ls[0]  
    elif ls[0] < ls[1]:  
        return ls[0]  
    else:  
        return fad(ls[1:])  
  
def gad(ls):  
    if len(ls) == 1:  
        return ls[0]  
    elif ls[0] < ls[1]:  
        return gad(ls[1:])  
    else:  
        return ls[0]  
  
print(fad([6, 8, 10, 7, 5]))  
print(gad([6, 8, 10, 7, 5]))
```

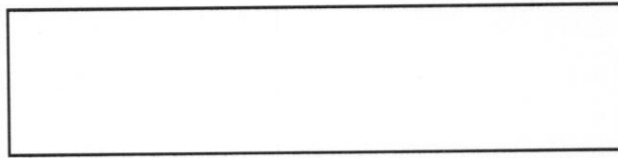
8. מה תדפיס התוכנית הבאה?

```
class FOO:

    def __init__(self):
        FOO.x = [3, 1]
        self.y = [9, 4]

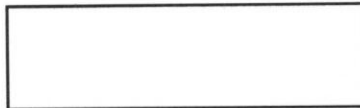
    def g(self):
        FOO.x[1] = FOO.x[1] + 7
        self.y *= 2
        return FOO.x + self.y

a,b = FOO(), FOO()
print(a.g())
print(a.g())
print(b.g())
```



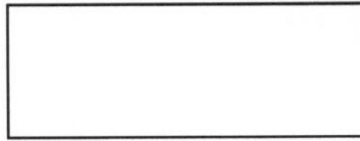
9. מה תדפיס התוכנית הבאה?

```
alist = ["14", 7, [14, 7], 14]
try:
    for item in alist:
        try:
            print(item + int(item), end = " ")
        except(TypeError): print("a", end = " ")
        finally: print("b", end = " ")
except: print("")
finally: print("b", end = " ")
```



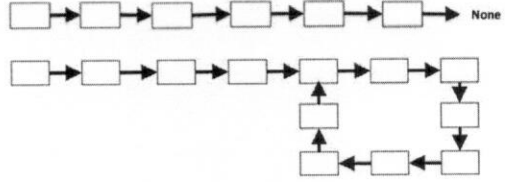
10. מה תדפיס התוכנית הבאה?

```
def foo(word, i):  
    if i >= len(word):  
        return  
    yield from foo(word, i+1)  
    yield word[i]  
  
for j in foo("intro", 0): print(j, end = "")
```



2. נתונה מחלקה עבור קודקוד ברשימה מקושרת.

```
class Node():
    def __init__(self, next=None):
        self.next = next
```



באזור למעלה מופיעות שתי רשימות מקושרות. אחת עם מעגל ואחת ללא מעגל. כתבו פונקציה `find_cycle(head)` המקבלת מצביע לראש של רשימה ומחזירה `True` במקרה שהרשימה מכילה מעגל ו-`False` אם הרשימה אינה מכילה מעגל (הרשימה עשויה להיות גם ריקה ואז אין בה מעגל).

מגבלות: מותר להשתמש רק בכמות קבועה של זיכרון (שאינה תלויה במספר הקודקודים ברשימה).
 אין לשנות את הרשימה המקורית. שימו לב: ניתן לפתור את הבעיה בסיבוכיות $O(n)$ אך גם פתרונות בסיבוכיות $O(n^2)$ יוכלו לקבל ניקוד מלא.

```
def find_cycle(head):
```



```
def prime_components(n):
```

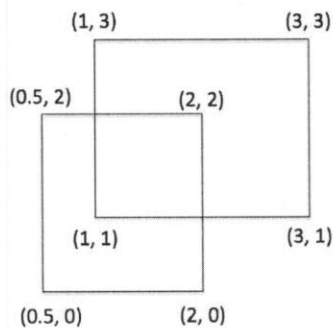
הנחה: נתון שבתא הראשון ברשימה $1st[0]$ מופיע תו סוגריים פותח (") ואילו בתא האחרון שלה $1st[-1]$ יש תו סוגר " ("). מכאן נובע גם שיש ברשימה לפחות שני תאים.

דרישות נוספות: יש לפתור את הבעיה ביעילות. אלגוריתם הרץ בזמן $O(n)$ יקבל רק חלק קטן מאוד מהנקודות.

```
def find_pair(lst):
```

6. כתבו מחלקה Rect המייצגת מלבן שצלעותיו מקבילות לצירים. עליכם לממש את ההתנהגות הבאה:

- הבנאי של המחלקה `__init__(pt1, pt2)` מקבל שתי נקודות המייצגות פינות מנוגדות (באלכסון זו מזו) של המלבן. שימו לב שיש שני אלכסונים אפשריים במלבן. ניתן לתת את נקודות התואמות לכל אחד מהאלכסונים, בכל סדר שהוא.
- כל נקודה מיוצגת ע"י רשימה באורך 2 (עבור קואורדינטת ה-x וקואורדינטת ה-y של הנקודה).
- למחלקה יש מתודה `overlap_rect(self, other)` המקבלת מלבן נוסף (`other`) ומחזירה מלבן חדש המתקבל מחיתוך המלבן הנוכחי ו-`other`. אם המלבנים אינם חופפים יש להחזיר `None`.



לדוגמא, התוכנית הבאה

```
a = Rect([0.5, 0], [2, 2])
b = Rect([3, 1], [1, 3])
```

תייצר את שני המלבנים בתמונה משמאל. אחרי כן, כתיבת

```
c = a.overlap_rect(b)
```

תייצר מלבן חדש שפינותיו `[1,1]` ו-`[2,2]`.

ניתן להניח שפינות המלבן הן חוקיות ושהקואורדינטות שלהן חיוביות.

שימו לב שיתכן ששטחו של מלבן הוא 0 אם לנקודות `pt1, pt2` יש את

אותה קואורדינטת x ו/או את אותה קואורדינטת y.

```
class Rect():
```

ניתן להמשיך בעמוד הבא

